Agenda: Modules and packages

1. Q&A
2. What are modules? What do they give us?
3. The various forms of the `import` statement
4. Develop a simple module
5. What happens when we import a module?
6. The special `__name__` variable, and what it means
7. Python's standard library
8. PyPI and third-party packages
9. `uv` and `pip` for installing packages from PyPI
10. AMA -- ask me anything
11. What's next?

What are modules?

We've discussed the DRY ("don't repeat yourself") rule for programming:

- If we have roughly the same code, several lines in a row, then we can replace them with a loop.
- If we have roughly the same functionality in several places in a program, we can write a function, and then invoke it wherever we want
- if we have roughly the same functionality in several different programs, we can store the functions and/or data in a *library*.

In Python, we call our libraries "modules." (A module is a single file containing Python code; if you have more than one file in your library, we in the Python world call it a "package.")

Modules in Python give us at least three types of functionality:

1. They contain the data and the functions we might want to use.
2. They provide us with *namespaces*, separations between different sets of defined names.
3. Modules (and packages) allow us to distribute our code to other people, working on other programs.

Modules are among the most important and frequently used pieces of a Python program. They're complicated under the hood, but using them is very easy, and writing simple modules is also very easy.

Let's use a module!

If you want to use a module, you use the `import` statement. `import` takes the name of a module as its argument, uses that to load a module from disk (from the file where it's defined), creates a module in Python's memory, and assigns that module to a variable.

In [1]:
```
import random
```

What do we get from the above line?

1. `import` is not a function. Don't invoke it with `()`. Rather, just say `import` and then after a space, put the name of the module you want to load and then define.
2. The name that we put after `import` is *not* a string, and not a filename. Rather, it's the name of the variable we want to create, *plus* the name of the file we want to load. In this case, we're telling Python to load `random.py`, where `py` is the suffix for most Python programs and all Python modules.
3. Many other languages have similar functionality to `import`, but there you often pass a string or a filename, and that's what is loaded. That's not how it works here.

After our `import` has run, the `random` variable is defined, and it contains a module called `random`, with all of the data and functionality that were defined in that module.

In [2]:
```
type(random)  # what kind of object, or value, do we have in the "random" va
```

Out[2]:
```
module
```

In [3]:
```
random   # show me the module's printed representation
```

Out[3]:
```
<module 'random' from '/Users/reuven/.local/share/uv/python/cpython-3.14.7-
macos-aarch64-none/lib/python3.14/random.py'>
```

Using names from modules

The module itself is just a warehouse for names. If you want to retrieve a value from a module, just name module, then `.`, then the name you want:

```
random.randint
```

This retrieves the value. But if the value is a function, you'll still need to provide `()` and (potentially) one or more arguments.

If it's just data, then you can say

```
random.NAME
```

How can I know what names are defined?

1. Use `dir(random)` to find out what names are defined on the `random` module. Those names, when put after a `.`, are all available on the module object.
2. You can use `help(random)` to get the docstring (the documentation from functions -- also on modules!) that often does a good job of not only listing the names, but showing the documentation
3. You can go to `docs.python.org`, where the standard library is documented.

In [4]:
```
dir(random)
```

Out[4]:
```
['BPF',
 'LOG4',
 'NV_MAGICCONST',
 'RECIP_BPF',
 'Random',
 'SG_MAGICCONST',
 'SystemRandom',
 'TWOPI',
 '_ONE',
 '_Sequence',
 '__all__',
 '__builtins__',
 '__cached__',
 '__doc__',
 '__file__',
 '__loader__',
 '__name__',
 '__package__',
 '__spec__',
 '_accumulate',
 '_acos',
 '_bisect',
 '_ceil',
 '_cos',
 '_e',
 '_exp',
 '_fabs',
 '_floor',
 '_index',
 '_inst',
 '_isfinite',
 '_lgamma',
 '_log',
 '_log2',
 '_os',
 '_parse_args',
 '_pi',
 '_random',
 '_repeat',
 '_sha512',
 '_sin',
 '_sqrt',
 '_test',
 '_test_generator',
 '_urandom',
 'betavariate',
 'binomialvariate',
 'choice',
 'choices',
 'expovariate',
 'gammavariate',
 'gauss',
 'getrandbits',
 'getstate',
 'lognormvariate',
 'main',
```

```
 'normalvariate',
 'paretovariate',
 'randbytes',
 'randint',
 'random',
 'randrange',
 'sample',
 'seed',
 'setstate',
 'shuffle',
 'triangular',
 'uniform',
 'vonmisesvariate',
 'weibullvariate']
```

In [5]:
```
help(random)
```

```
Help on module random:

NAME
    random - Random variable generators.

MODULE REFERENCE
    https://docs.python.org/3.14/library/random#module-random

    The following documentation is automatically generated from the Python
    source files.  It may be incomplete, incorrect or include features that
    are considered implementation detail and may vary between Python
    implementations.  When in doubt, consult the module reference at the
    location listed above.

DESCRIPTION
        bytes
        -----
               uniform bytes (values between 0 and 255)

        integers
        --------
               uniform within range

        sequences
        ---------
               pick random element
               pick random sample
               pick weighted random sample
               generate random permutation

        distributions on the real line:
        ------------------------------
               uniform
               triangular
               normal (Gaussian)
               lognormal
               negative exponential
               gamma
               beta
               pareto
               Weibull

        distributions on the circle (angles 0 to 2pi)
        ---------------------------------------------
               circular uniform
               von Mises

        discrete distributions
        ----------------------
               binomial


    General notes on the underlying Mersenne Twister core generator:

    * The period is 2**19937-1.
    * It is one of the most extensively tested generators in existence.
```

```
    * The random() method is implemented in C, executes in a single Python s
tep,
      and is, therefore, threadsafe.

CLASSES
    _random.Random(builtins.object)
        Random
            SystemRandom

    class Random(_random.Random)
     |  Random(x=None)
     |
     |  Random number generator base class used by bound module functions.
     |
     |  Used to instantiate instances of Random to get generators that don't
     |  share state.
     |
     |  Class Random can also be subclassed if you want to use a different b
asic
     |  generator of your own devising: in that case, override the following
     |  methods:  random(), seed(), getstate(), and setstate().
     |  Optionally, implement a getrandbits() method so that randrange()
     |  can cover arbitrarily large ranges.
     |
     |  Method resolution order:
     |      Random
     |      _random.Random
     |      builtins.object
     |
     |  Methods defined here:
     |
     |  __getstate__(self)
     |      Helper for pickle.
     |
     |  __init__(self, x=None)
     |      Initialize an instance.
     |
     |      Optional argument x controls seeding, as for Random.seed().
     |
     |  __reduce__(self)
     |      Helper for pickle.
     |
     |  __setstate__(self, state)
     |
     |  betavariate(self, alpha, beta)
     |      Beta distribution.
     |
     |      Conditions on the parameters are alpha > 0 and beta > 0.
     |      Returned values range between 0 and 1.
     |
     |      The mean (expected value) and variance of the random variable ar
e:
     |
     |          E[X] = alpha / (alpha + beta)
     |          Var[X] = alpha * beta / ((alpha + beta)**2 * (alpha + beta +
1))
```

```
      |
      |  binomialvariate(self, n=1, p=0.5)
      |      Binomial random variable.
      |
      |      Gives the number of successes for *n* independent trials
      |      with the probability of success in each trial being *p*:
      |
      |          sum(random() < p for i in range(n))
      |
      |      Returns an integer in the range:
      |
      |          0 <= X <= n
      |
      |      The integer is chosen with the probability:
      |
      |          P(X == k) = math.comb(n, k) * p ** k * (1 - p) ** (n - k)
      |
      |      The mean (expected value) and variance of the random variable ar
e:
      |
      |          E[X] = n * p
      |          Var[X] = n * p * (1 - p)
      |
      |  choice(self, seq)
      |      Choose a random element from a non-empty sequence.
      |
      |  choices(self, population, weights=None, *, cum_weights=None, k=1)
      |      Return a k sized list of population elements chosen with replace
ment.
      |
      |      If the relative weights or cumulative weights are not specified,
      |      the selections are made with equal probability.
      |
      |  expovariate(self, lambd=1.0)
      |      Exponential distribution.
      |
      |      lambd is 1.0 divided by the desired mean.  It should be
      |      nonzero.  (The parameter would be called "lambda", but that is
      |      a reserved word in Python.)  Returned values range from 0 to
      |      positive infinity if lambd is positive, and from negative
      |      infinity to 0 if lambd is negative.
      |
      |      The mean (expected value) and variance of the random variable ar
e:
      |
      |          E[X] = 1 / lambd
      |          Var[X] = 1 / lambd ** 2
      |
      |  gammavariate(self, alpha, beta)
      |      Gamma distribution.  Not the gamma function!
      |
      |      Conditions on the parameters are alpha > 0 and beta > 0.
      |
      |      The probability distribution function is:
      |
      |                  x ** (alpha - 1) * math.exp(-x / beta)
```

```
      |        pdf(x) =  --------------------------------------
      |                     math.gamma(alpha) * beta ** alpha
      |
      |      The mean (expected value) and variance of the random variable ar
e:
      |
      |          E[X] = alpha * beta
      |          Var[X] = alpha * beta ** 2
      |
      |  gauss(self, mu=0.0, sigma=1.0)
      |      Gaussian distribution.
      |
      |      mu is the mean, and sigma is the standard deviation.  This is
      |      slightly faster than the normalvariate() function.
      |
      |      Not thread-safe without a lock around calls.
      |
      |  getstate(self)
      |      Return internal state; can be passed to setstate() later.
      |
      |  lognormvariate(self, mu, sigma)
      |      Log normal distribution.
      |
      |      If you take the natural logarithm of this distribution, you'll g
et a
      |      normal distribution with mean mu and standard deviation sigma.
      |      mu can have any value, and sigma must be greater than zero.
      |
      |  normalvariate(self, mu=0.0, sigma=1.0)
      |      Normal distribution.
      |
      |      mu is the mean, and sigma is the standard deviation.
      |
      |  paretovariate(self, alpha)
      |      Pareto distribution.  alpha is the shape parameter.
      |
      |  randbytes(self, n)
      |      Generate n random bytes.
      |
      |  randint(self, a, b)
      |      Return random integer in range [a, b], including both end point
s.
      |
      |  randrange(self, start, stop=None, step=1)
      |      Choose a random item from range(stop) or range(start, stop[, ste
p]).
      |
      |      Roughly equivalent to ``choice(range(start, stop, step))`` but
      |      supports arbitrarily large ranges and is optimized for common ca
ses.
      |
      |  sample(self, population, k, *, counts=None)
      |      Chooses k unique random elements from a population sequence.
      |
      |      Returns a new list containing elements from the population while
      |      leaving the original population unchanged.  The resulting list i
```

```
s
      |      in selection order so that all sub-slices will also be valid ran
dom
      |      samples.  This allows raffle winners (the sample) to be partitio
ned
      |      into grand prize and second place winners (the subslices).
      |
      |      Members of the population need not be hashable or unique.  If th
e
      |      population contains repeats, then each occurrence is a possible
      |      selection in the sample.
      |
      |      Repeated elements can be specified one at a time or with the opt
ional
      |      counts parameter.  For example:
      |
      |          sample(['red', 'blue'], counts=[4, 2], k=5)
      |
      |      is equivalent to:
      |
      |          sample(['red', 'red', 'red', 'red', 'blue', 'blue'], k=5)
      |
      |      To choose a sample from a range of integers, use range() for the
      |      population argument.  This is especially fast and space efficien
t
      |      for sampling from a large population:
      |
      |          sample(range(10000000), 60)
      |
      |  seed(self, a=None, version=2)
      |      Initialize internal state from a seed.
      |
      |      The only supported seed types are None, int, float,
      |      str, bytes, and bytearray.
      |
      |      None or no argument seeds from current time or from an operating
      |      system specific randomness source if available.
      |
      |      If *a* is an int, all bits are used.
      |
      |      For version 2 (the default), all of the bits are used if *a* is
a str,
      |      bytes, or bytearray.  For version 1 (provided for reproducing ra
ndom
      |      sequences from older versions of Python), the algorithm for str
and
      |      bytes generates a narrower range of seeds.
      |
      |  setstate(self, state)
      |      Restore internal state from object returned by getstate().
      |
      |  shuffle(self, x)
      |      Shuffle list x in place, and return None.
      |
      |  triangular(self, low=0.0, high=1.0, mode=None)
      |      Triangular distribution.
```

```
      |
      |      Continuous distribution bounded by given lower and upper limits,
      |      and having a given mode value in-between.
      |
      |      http://en.wikipedia.org/wiki/Triangular_distribution
      |
      |      The mean (expected value) and variance of the random variable ar
e:
      |
      |          E[X] = (low + high + mode) / 3
      |          Var[X] = (low**2 + high**2 + mode**2 - low*high - low*mode -
high*mode) / 18
      |
      |  uniform(self, a, b)
      |      Get a random number in the range [a, b) or [a, b] depending on r
ounding.
      |
      |      The mean (expected value) and variance of the random variable ar
e:
      |
      |          E[X] = (a + b) / 2
      |          Var[X] = (b - a) ** 2 / 12
      |
      |  vonmisesvariate(self, mu, kappa)
      |      Circular data distribution.
      |
      |      mu is the mean angle, expressed in radians between 0 and 2*pi, a
nd
      |      kappa is the concentration parameter, which must be greater than
or
      |      equal to zero.  If kappa is equal to zero, this distribution red
uces
      |      to a uniform random angle over the range 0 to 2*pi.
      |
      |  weibullvariate(self, alpha, beta)
      |      Weibull distribution.
      |
      |      alpha is the scale parameter and beta is the shape parameter.
      |
      |  ----------------------------------------------------------------------
--
      |  Class methods defined here:
      |
      |  __init_subclass__(**kwargs)
      |      Control how subclasses generate random integers.
      |
      |      The algorithm a subclass can use depends on the random() and/or
      |      getrandbits() implementation available to it and determines
      |      whether it can generate random integers from arbitrarily large
      |      ranges.
      |
      |  ----------------------------------------------------------------------
--
      |  Data descriptors defined here:
      |
      |  __dict__
```

```
      |      dictionary for instance variables
      |
      |  __weakref__
      |      list of weak references to the object
      |
      |  ----------------------------------------------------------------------
--
      |  Data and other attributes defined here:
      |
      |  VERSION = 3
      |
      |  ----------------------------------------------------------------------
--
      |  Methods inherited from _random.Random:
      |
      |  getrandbits(self, k, /)
      |      getrandbits(k) -> x.  Generates an int with k random bits.
      |
      |  random(self, /)
      |      random() -> x in the interval [0, 1).
      |
      |  ----------------------------------------------------------------------
--
      |  Static methods inherited from _random.Random:
      |
      |  __new__(*args, **kwargs) class method of _random.Random
      |      Create and return a new object.  See help(type) for accurate sig
nature.

    class SystemRandom(Random)
      |  SystemRandom(x=None)
      |
      |  Alternate random number generator using sources provided
      |  by the operating system (such as /dev/urandom on Unix or
      |  CryptGenRandom on Windows).
      |
      |   Not available on all systems (see os.urandom() for details).
      |
      |  Method resolution order:
      |      SystemRandom
      |      Random
      |      _random.Random
      |      builtins.object
      |
      |  Methods defined here:
      |
      |  getrandbits(self, k)
      |      getrandbits(k) -> x.  Generates an int with k random bits.
      |
      |  getstate = _notimplemented(self, *args, **kwds)
      |
      |  randbytes(self, n)
      |      Generate n random bytes.
      |
      |  random(self)
      |      Get the next random number in the range 0.0 <= X < 1.0.
```

```
      |
      |  seed(self, *args, **kwds)
      |      Stub method.  Not used for a system random number generator.
      |
      |  setstate = _notimplemented(self, *args, **kwds)
      |
      |  ----------------------------------------------------------------------
--
      |  Methods inherited from Random:
      |
      |  __getstate__(self)
      |      Helper for pickle.
      |
      |  __init__(self, x=None)
      |      Initialize an instance.
      |
      |      Optional argument x controls seeding, as for Random.seed().
      |
      |  __reduce__(self)
      |      Helper for pickle.
      |
      |  __setstate__(self, state)
      |
      |  betavariate(self, alpha, beta)
      |      Beta distribution.
      |
      |      Conditions on the parameters are alpha > 0 and beta > 0.
      |      Returned values range between 0 and 1.
      |
      |      The mean (expected value) and variance of the random variable ar
e:
      |
      |          E[X] = alpha / (alpha + beta)
      |          Var[X] = alpha * beta / ((alpha + beta)**2 * (alpha + beta +
1))
      |
      |  binomialvariate(self, n=1, p=0.5)
      |      Binomial random variable.
      |
      |      Gives the number of successes for *n* independent trials
      |      with the probability of success in each trial being *p*:
      |
      |          sum(random() < p for i in range(n))
      |
      |      Returns an integer in the range:
      |
      |          0 <= X <= n
      |
      |      The integer is chosen with the probability:
      |
      |          P(X == k) = math.comb(n, k) * p ** k * (1 - p) ** (n - k)
      |
      |      The mean (expected value) and variance of the random variable ar
e:
      |
      |          E[X] = n * p
```

```
      |          Var[X] = n * p * (1 - p)
      |
      |  choice(self, seq)
      |      Choose a random element from a non-empty sequence.
      |
      |  choices(self, population, weights=None, *, cum_weights=None, k=1)
      |      Return a k sized list of population elements chosen with replace
ment.
      |
      |      If the relative weights or cumulative weights are not specified,
      |      the selections are made with equal probability.
      |
      |  expovariate(self, lambd=1.0)
      |      Exponential distribution.
      |
      |      lambd is 1.0 divided by the desired mean.  It should be
      |      nonzero.  (The parameter would be called "lambda", but that is
      |      a reserved word in Python.)  Returned values range from 0 to
      |      positive infinity if lambd is positive, and from negative
      |      infinity to 0 if lambd is negative.
      |
      |      The mean (expected value) and variance of the random variable ar
e:
      |
      |          E[X] = 1 / lambd
      |          Var[X] = 1 / lambd ** 2
      |
      |  gammavariate(self, alpha, beta)
      |      Gamma distribution.  Not the gamma function!
      |
      |      Conditions on the parameters are alpha > 0 and beta > 0.
      |
      |      The probability distribution function is:
      |
      |                  x ** (alpha - 1) * math.exp(-x / beta)
      |        pdf(x) =  --------------------------------------
      |                    math.gamma(alpha) * beta ** alpha
      |
      |      The mean (expected value) and variance of the random variable ar
e:
      |
      |          E[X] = alpha * beta
      |          Var[X] = alpha * beta ** 2
      |
      |  gauss(self, mu=0.0, sigma=1.0)
      |      Gaussian distribution.
      |
      |      mu is the mean, and sigma is the standard deviation.  This is
      |      slightly faster than the normalvariate() function.
      |
      |      Not thread-safe without a lock around calls.
      |
      |  lognormvariate(self, mu, sigma)
      |      Log normal distribution.
      |
      |      If you take the natural logarithm of this distribution, you'll g
```

```
et a
      |      normal distribution with mean mu and standard deviation sigma.
      |      mu can have any value, and sigma must be greater than zero.
      |
      |  normalvariate(self, mu=0.0, sigma=1.0)
      |      Normal distribution.
      |
      |      mu is the mean, and sigma is the standard deviation.
      |
      |  paretovariate(self, alpha)
      |      Pareto distribution.  alpha is the shape parameter.
      |
      |  randint(self, a, b)
      |      Return random integer in range [a, b], including both end point
s.
      |
      |  randrange(self, start, stop=None, step=1)
      |      Choose a random item from range(stop) or range(start, stop[, ste
p]).
      |
      |      Roughly equivalent to ``choice(range(start, stop, step))`` but
      |      supports arbitrarily large ranges and is optimized for common ca
ses.
      |
      |  sample(self, population, k, *, counts=None)
      |      Chooses k unique random elements from a population sequence.
      |
      |      Returns a new list containing elements from the population while
      |      leaving the original population unchanged.  The resulting list i
s
      |      in selection order so that all sub-slices will also be valid ran
dom
      |      samples.  This allows raffle winners (the sample) to be partitio
ned
      |      into grand prize and second place winners (the subslices).
      |
      |      Members of the population need not be hashable or unique.  If th
e
      |      population contains repeats, then each occurrence is a possible
      |      selection in the sample.
      |
      |      Repeated elements can be specified one at a time or with the opt
ional
      |      counts parameter.  For example:
      |
      |          sample(['red', 'blue'], counts=[4, 2], k=5)
      |
      |      is equivalent to:
      |
      |          sample(['red', 'red', 'red', 'red', 'blue', 'blue'], k=5)
      |
      |      To choose a sample from a range of integers, use range() for the
      |      population argument.  This is especially fast and space efficien
t
      |      for sampling from a large population:
      |
```

```
      |          sample(range(10000000), 60)
      |
      |  shuffle(self, x)
      |      Shuffle list x in place, and return None.
      |
      |  triangular(self, low=0.0, high=1.0, mode=None)
      |      Triangular distribution.
      |
      |      Continuous distribution bounded by given lower and upper limits,
      |      and having a given mode value in-between.
      |
      |      http://en.wikipedia.org/wiki/Triangular_distribution
      |
      |      The mean (expected value) and variance of the random variable ar
e:
      |
      |          E[X] = (low + high + mode) / 3
      |          Var[X] = (low**2 + high**2 + mode**2 - low*high - low*mode -
high*mode) / 18
      |
      |  uniform(self, a, b)
      |      Get a random number in the range [a, b) or [a, b] depending on r
ounding.
      |
      |      The mean (expected value) and variance of the random variable ar
e:
      |
      |          E[X] = (a + b) / 2
      |          Var[X] = (b - a) ** 2 / 12
      |
      |  vonmisesvariate(self, mu, kappa)
      |      Circular data distribution.
      |
      |      mu is the mean angle, expressed in radians between 0 and 2*pi, a
nd
      |      kappa is the concentration parameter, which must be greater than
or
      |      equal to zero.  If kappa is equal to zero, this distribution red
uces
      |      to a uniform random angle over the range 0 to 2*pi.
      |
      |  weibullvariate(self, alpha, beta)
      |      Weibull distribution.
      |
      |      alpha is the scale parameter and beta is the shape parameter.
      |
      |  ----------------------------------------------------------------------
--
      |  Class methods inherited from Random:
      |
      |  __init_subclass__(**kwargs)
      |      Control how subclasses generate random integers.
      |
      |      The algorithm a subclass can use depends on the random() and/or
      |      getrandbits() implementation available to it and determines
      |      whether it can generate random integers from arbitrarily large
```

```
      |      ranges.
      |
      |  ----------------------------------------------------------------------
--
      |  Data descriptors inherited from Random:
      |
      |  __dict__
      |      dictionary for instance variables
      |
      |  __weakref__
      |      list of weak references to the object
      |
      |  ----------------------------------------------------------------------
--
      |  Data and other attributes inherited from Random:
      |
      |  VERSION = 3
      |
      |  ----------------------------------------------------------------------
--
      |  Static methods inherited from _random.Random:
      |
      |  __new__(*args, **kwargs) class method of _random.Random
      |      Create and return a new object.  See help(type) for accurate sig
nature.

FUNCTIONS
    betavariate(alpha, beta) method of Random instance
        Beta distribution.

        Conditions on the parameters are alpha > 0 and beta > 0.
        Returned values range between 0 and 1.

        The mean (expected value) and variance of the random variable are:

            E[X] = alpha / (alpha + beta)
            Var[X] = alpha * beta / ((alpha + beta)**2 * (alpha + beta + 1))

    binomialvariate(n=1, p=0.5) method of Random instance
        Binomial random variable.

        Gives the number of successes for *n* independent trials
        with the probability of success in each trial being *p*:

            sum(random() < p for i in range(n))

        Returns an integer in the range:

            0 <= X <= n

        The integer is chosen with the probability:

            P(X == k) = math.comb(n, k) * p ** k * (1 - p) ** (n - k)

        The mean (expected value) and variance of the random variable are:
```

```
            E[X] = n * p
            Var[X] = n * p * (1 - p)

    choice(seq) method of Random instance
        Choose a random element from a non-empty sequence.

    choices(population, weights=None, *, cum_weights=None, k=1) method of Ra
ndom instance
        Return a k sized list of population elements chosen with replacemen
t.

        If the relative weights or cumulative weights are not specified,
        the selections are made with equal probability.

    expovariate(lambd=1.0) method of Random instance
        Exponential distribution.

        lambd is 1.0 divided by the desired mean.  It should be
        nonzero.  (The parameter would be called "lambda", but that is
        a reserved word in Python.)  Returned values range from 0 to
        positive infinity if lambd is positive, and from negative
        infinity to 0 if lambd is negative.

        The mean (expected value) and variance of the random variable are:

            E[X] = 1 / lambd
            Var[X] = 1 / lambd ** 2

    gammavariate(alpha, beta) method of Random instance
        Gamma distribution.  Not the gamma function!

        Conditions on the parameters are alpha > 0 and beta > 0.

        The probability distribution function is:

                    x ** (alpha - 1) * math.exp(-x / beta)
          pdf(x) =  --------------------------------------
                      math.gamma(alpha) * beta ** alpha

        The mean (expected value) and variance of the random variable are:

            E[X] = alpha * beta
            Var[X] = alpha * beta ** 2

    gauss(mu=0.0, sigma=1.0) method of Random instance
        Gaussian distribution.

        mu is the mean, and sigma is the standard deviation.  This is
        slightly faster than the normalvariate() function.

        Not thread-safe without a lock around calls.

    getrandbits(k, /) method of Random instance
        getrandbits(k) -> x.  Generates an int with k random bits.

    getstate() method of Random instance
```

```
        Return internal state; can be passed to setstate() later.

    lognormvariate(mu, sigma) method of Random instance
        Log normal distribution.

        If you take the natural logarithm of this distribution, you'll get a
        normal distribution with mean mu and standard deviation sigma.
        mu can have any value, and sigma must be greater than zero.

    normalvariate(mu=0.0, sigma=1.0) method of Random instance
        Normal distribution.

        mu is the mean, and sigma is the standard deviation.

    paretovariate(alpha) method of Random instance
        Pareto distribution.  alpha is the shape parameter.

    randbytes(n) method of Random instance
        Generate n random bytes.

    randint(a, b) method of Random instance
        Return random integer in range [a, b], including both end points.

    random() method of Random instance
        random() -> x in the interval [0, 1).

    randrange(start, stop=None, step=1) method of Random instance
        Choose a random item from range(stop) or range(start, stop[, step]).

        Roughly equivalent to ``choice(range(start, stop, step))`` but
        supports arbitrarily large ranges and is optimized for common cases.

    sample(population, k, *, counts=None) method of Random instance
        Chooses k unique random elements from a population sequence.

        Returns a new list containing elements from the population while
        leaving the original population unchanged.  The resulting list is
        in selection order so that all sub-slices will also be valid random
        samples.  This allows raffle winners (the sample) to be partitioned
        into grand prize and second place winners (the subslices).

        Members of the population need not be hashable or unique.  If the
        population contains repeats, then each occurrence is a possible
        selection in the sample.

        Repeated elements can be specified one at a time or with the optiona
l
        counts parameter.  For example:

            sample(['red', 'blue'], counts=[4, 2], k=5)

        is equivalent to:

            sample(['red', 'red', 'red', 'red', 'blue', 'blue'], k=5)

        To choose a sample from a range of integers, use range() for the
```

```
        population argument.  This is especially fast and space efficient
        for sampling from a large population:

            sample(range(10000000), 60)

    seed(a=None, version=2) method of Random instance
        Initialize internal state from a seed.

        The only supported seed types are None, int, float,
        str, bytes, and bytearray.

        None or no argument seeds from current time or from an operating
        system specific randomness source if available.

        If *a* is an int, all bits are used.

        For version 2 (the default), all of the bits are used if *a* is a st
r,
        bytes, or bytearray.  For version 1 (provided for reproducing random
        sequences from older versions of Python), the algorithm for str and
        bytes generates a narrower range of seeds.

    setstate(state) method of Random instance
        Restore internal state from object returned by getstate().

    shuffle(x) method of Random instance
        Shuffle list x in place, and return None.

    triangular(low=0.0, high=1.0, mode=None) method of Random instance
        Triangular distribution.

        Continuous distribution bounded by given lower and upper limits,
        and having a given mode value in-between.

        http://en.wikipedia.org/wiki/Triangular_distribution

        The mean (expected value) and variance of the random variable are:

            E[X] = (low + high + mode) / 3
            Var[X] = (low**2 + high**2 + mode**2 - low*high - low*mode - hig
h*mode) / 18

    uniform(a, b) method of Random instance
        Get a random number in the range [a, b) or [a, b] depending on round
ing.

        The mean (expected value) and variance of the random variable are:

            E[X] = (a + b) / 2
            Var[X] = (b - a) ** 2 / 12

    vonmisesvariate(mu, kappa) method of Random instance
        Circular data distribution.

        mu is the mean angle, expressed in radians between 0 and 2*pi, and
        kappa is the concentration parameter, which must be greater than or
```

```
        equal to zero.  If kappa is equal to zero, this distribution reduces
        to a uniform random angle over the range 0 to 2*pi.

    weibullvariate(alpha, beta) method of Random instance
        Weibull distribution.

        alpha is the scale parameter and beta is the shape parameter.

DATA
    __all__ = ['Random', 'SystemRandom', 'betavariate', 'binomialvariat
e',...

FILE
    /Users/reuven/.local/share/uv/python/cpython-3.14.7-macos-aarch64-none/l
ib/python3.14/random.py
```

In [11]:
```
# now that I have loaded up "random", I can invoke

random.randint(0, 5)  # randint takes two arguments -- the minimum, and the
```

Out[11]: 1

In this set of examples:

- `random` is the module name, and what we directly imported
- `randint` is the function that was defined in the module, which we are invoking.

Exercise: Guessing game

1. Generate a random `secret` from the `random` module, between 0 and 100.
2. Ask the user, repeatedly, to guess the number:
   - If they guess right, congratulate them and exit
   - If they guess too low, say "Too low!"
   - If they guess too high, say "Too high!"
3. If they got it wrong, they get to guess again.
4. Print the number of guesses when they're done.

In [16]:
```
import random

counter = 0
secret = random.randint(0, 100)

while True:
    guess_text = input('Guess: ').strip()

    if guess_text.isdigit():
        counter += 1
        guess = int(guess_text)
        if guess == secret:
```

```
            print(f'You got it in {counter} guesses.')
            break

        elif guess < secret:
            print('Too low!')

        else:
            print('Too high!')

    else:
        print(f'{guess_text} is not numeric; try again')
```

```
Too low!
Too high!
Too high!
Too high!
You got it in 5 guesses.
```

Where did the `random` module get loaded from?

In [17]:
```
random
```

Out[17]:
```
<module 'random' from '/Users/reuven/.local/share/uv/python/cpython-3.14.7-macos-aarch64-none/lib/python3.14/random.py'>
```

How did Python know to search there?

The `sys.path` variable -- which is in the `sys` module -- contains a list of strings, each a directory in which a file might appear.

In [18]:
```
import sys
sys.path
```

Out[18]:
```
['/Users/reuven/.local/share/uv/python/cpython-3.14.7-macos-aarch64-none/lib/python314.zip',
 '/Users/reuven/.local/share/uv/python/cpython-3.14.7-macos-aarch64-none/lib/python3.14',
 '/Users/reuven/.local/share/uv/python/cpython-3.14.7-macos-aarch64-none/lib/python3.14/lib-dynload',
 '',
 '/Users/reuven/Courses/Current/OReilly-Python-Oct2026-2026-10/.venv/lib/python3.14/site-packages']
```

In [19]:
```
import asdfafajfdadfa
```

```
---------------------------------------------------------------------------
ModuleNotFoundError                       Traceback (most recent call last)
Cell In[19], line 1
----> 1 import asdfafajfdadfa

ModuleNotFoundError: No module named 'asdfafajfdadfa'
```

Shorthand

You can, of course, keep saying `random.randint` inside of your program. But at a certain point, it becomes to loud, too long, and too annoying for us to write. We know that `randdin` is from `random`.

Sometimes -- for easier writing, easier maintainability -- we want to just use the function name, not the name. Why should I keep writing `random.randint`, when I want to just write `randint`?

That won't work because `randint` isn't defined as a varible. It's an attribute on the module `random`. You can't normally just give the method name; you need to specify the module, too.

But ... what if I want tod?

I can use an alternative syntax:

```
from MODULE import FUNCTION
```

In [20]:
```
randint(0, 100)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[20], line 1
----> 1 randint(0, 100)

NameError: name 'randint' is not defined
```

In [21]:
```
random.randint(0, 100)
```

Out[21]: 71

In [22]:
```
from random import randint

randint(0, 100)
```

Out[22]: 96

What does `from .. import` do?

1. It loads the entire module into memory.
2. But it only exposes and defines the one that you specify.

Many *many* people believe that `from .. import` saves memory, because it ony imports a handful of files. That's not true -- when you say `from .. import`, Python needs to find, then define, just one name -- to do that, it needs all of the names, and the entire module, loaded.

Variations on `import`

- `import MODNAME` -- defines the module variable
- `from MODNAME import NAME` -- defines just the names that we want to import
- `import MODNAME as ALIAS` -- loads the module, but changes the name of the variable it defines
- `from MODNAME import NAME as ALIAS` -- load the name ou want, and immediately change the name

Exercise: Another (weirder) guessing game

1. Define a list of strings, where each string is the name of the month of the year. (All 12!)
2. Use `random.choice`, a function in `random`, to select one string. Normally, you can invoke `random.choice` and get back one random value from that list. However, I don't want you to use `random.choice` directlyu. Instead, invoke it as `ch`, and set up the aliases to work.
3. Ask the user, repeatedly, to guess the secret string
4. When they answer correctly, tell them how many guesses they need.

In [23]:
```
random.choice(['a', 'b', 'c'])
```

Out[23]:
```
'c'
```

In [32]:
```
months = 'Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec'.split()
guess_counter = 0
secret = random.choice(months)

while True:
    guess_counter += 1
    guess = input('Enter a guess: ').strip()

    if guess == secret:
        print(f'You got it in {guess_counter} guesses')
        break
```

```
You got it in 9 guesses
```

Next up:

- Module names and alises
- Writing a simple module of our own

In [33]:
```python
# try again, but with the alias!

from random import choice as ch

months = 'Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec'.split()
guess_counter = 0
secret = ch(months)

while True:
    guess_counter += 1
    guess = input('Enter a guess: ').strip()

    if guess == secret:
        print(f'You got it in {guess_counter} guesses')
        break
```

```
You got it in 7 guesses
```

Why aliases?

1. Sometimes, it's a standard/convention. For example, the data-analysis Pandas library is always imported as `import panas as pd`. Why? Because everyone does.
2. You might otherwise conflict with another variable/name in the system -- or come so close that you don't want to risk it.
3. You can have nested hierarchies of names, such as `a.b.c.d.e.f`, and it's often easier to use `from import` and give an alias as well.

Names and values in Python

When I assign a value to a variable:

```
x = 5
```

I'm saying that the name `x` refers to the value `5`.

When we define a function:

```
def hello():
    return 'Hello!'
```

We're defining a function and then assigning that function value to the name `hello`.

If I import a module:

```
import random
```

Python is creating the module object/value, and then assigning it to the variable `random`.

In all three cases, we're assigning to a name ("identifier"), and Python doesn't really care whether it's an integer, string, dict, function, or module. In all cases, they're just values.

Many languages have one set of names for functions and another for data. That's not the case in Python -- any identifier can refer to any value, be it a function, module, or something else.

Can we write our own module?

Of course, the answer is *yes*!

So long as:

- The filename needs to end with `.py`
- The file needs to be in `sys.path` or in the same directory as the program importing it
- The file contains legit Python code -- usually variable assignments and function definitions

In [34]:
```
import mymod   # look for mymod.py in current directory, then sys.modules, a
```

In [35]:
```
# what names will be defined in mymod?

dir(mymod)
```

Out[35]:
```
['__builtins__',
 '__cached__',
 '__doc__',
 '__file__',
 '__loader__',
 '__name__',
 '__package__',
 '__spec__']
```

"Dunder" (short for "double underscore) is a way of marking names (of all sorts) as being special for Python -- either it's set by the language, or it's used by the language

internally, or if we define it, we change Python's behavior.

Bottom line is: Dunders are great, but only define them when you know what they do.

A few of the dunders in our module:

1. `__builtins__`, where the "builtin" names are stored - `len`, `str`, `dict`
2. `__file__`, a string containing the name of the file
3. `__name__`, the name of the module itself, should be identical to the variable we just defined

In [1]:
```
import mymod
```

In [2]:
```
dir(mymod)
```

Out[2]:
```
['__builtins__',
 '__cached__',
 '__doc__',
 '__file__',
 '__loader__',
 '__name__',
 '__package__',
 '__spec__',
 'hello',
 'x',
 'y']
```

In [3]:
```
mymod.hello
```

Out[3]:
```
<function mymod.hello(name)>
```

In [4]:
```
mymod.x
```

Out[4]:
```
10
```

In [5]:
```
mymod.y
```

Out[5]:
```
[100, 200, 300]
```

In [6]:
```
mymod.hello('Reuven')
```

Out[6]:
```
'Hello, Reuven, from mymod!'
```

Writing a module:

- Create a file whose name has a `.py` suffix, and which is in a directory in `sys.path`.
- In that file, put function and variable definitions
- A program can then `import` what you have defined

- The program can use the module's variables and functions by saying `.NAME`

Exercise: Shouter!

1. Write a module, `shout.py`, that has one function, `shout`. You might then neeed to run `shout.shout` because the module and function have the same name.
2. The function `shout` takes a string argument and returns it, but with several exclamation points after.
3. You should thus be able to say

`shout.shout('hello')` # returns 'hello!!!'

In [7]:
```
import shout
```

In [9]:
```
shout
```

Out[9]:
```
<module 'shout' from '/Users/reuven/Courses/Current/OReilly-Python-Oct2026-2026-10/shout.py'>
```

In [10]:
```
dir(shout)
```

Out[10]:
```
['__builtins__',
 '__cached__',
 '__doc__',
 '__file__',
 '__loader__',
 '__name__',
 '__package__',
 '__spec__',
 'shout']
```

In [11]:
```
shout.shout('hello, world')
```

Out[11]:
```
'hello, world!!!'
```

Exercise: Menu

The idea behind this module is that you might have multiple programs in which you'll want to let the user choose from among a number of options. This function will simplify that.

1. Create a new module, `menu.py`. This contains the single function `menu`. The `menu` function, when passed a list of strings, then asks the user to indicate which string they prefer.
2. If the user chooses a valid option from the menu, it is returned to the caller.
3. If the user chooses something invalid, they get an error message and are asked to try again

In [13]:
```
import menu
menu.menu(['a', 'b', 'c'])
```

```
Invalid choice; try one of ['a', 'b', 'c']
```

Out[13]:
```
'c'
```

In [17]:
```
# another way to reload is to invoke importlib.reload(menu)

import importlib
importlib.reload(menu)
```

Out[17]:
```
<module 'menu' from '/Users/reuven/Courses/Current/OReilly-Python-Oct2026-2026-10/menu.py'>
```

In [18]:
```
import menu
menu.menu(['a', 'b', 'c'])
```

```
Invalid choice; try one of ['a', 'b', 'c']
Invalid choice; try one of ['a', 'b', 'c']
Invalid choice; try one of ['a', 'b', 'c']
```

Out[18]:
```
'c'
```

Next up

1. What happens when you import a module?
2. The special `__name__` identifier
3. Python's standard library

What happens when you import a module?

It might seem very clear at first what happens when you use `import`:

- Python finds the file
- Python loads the file
- You now have the data from the file, as a module

It's more interesting and more complex than that -- we're not going to go into all of the details, but we should review some of them, because they do affect how your program works.

Finding the file is easy -- we already discussed `sys.path`. Python does a `for` loop on `sys.path` -- if you asked for `mymod`, it looks for `mymod.py` in each directory in `sys.path`. The important thing is: The first match wins. So if you put a module, e.g., `xml.py` in the current directory, you have now lost access to the `xml.py` that comes with Python! Because the current directory has priority over all directories in `sys.path`.

What does it mean for Python to "load" the file? For many people, it means that the definitions are somehow loaded into memory, but aren't actually defined until we need them.

This isn't true! It's far easier than this.

When you `import` a module, the module file is read and executed, line by line, immediately when you say `import MODULE`. If your module is very large, or takes a long time to load, then the program that imports it will be waiting on that line for a little while.

When the module is executed, it's done using a mechanism in Python that allows all of the definitions to be attached to a module. So the global variables defined in a module are turned into attributes on the module object.

- global `x` -> `mymod.x`
- global `y` -> `mymod.y`
- global `hello` -> `mymod.hello`

This seems to mean that we can put any code we want in a module, and it'll be executed when the module is imported! Can that be true?

In [1]:
```
import mymod
```

```
Welcome to mymod!
Thanks for visiting mymod!
```

In [2]:
```
dir(mymod)
```

Out[2]:
```
['__builtins__',
 '__cached__',
 '__doc__',
 '__file__',
 '__loader__',
 '__name__',
 '__package__',
 '__spec__',
 'hello',
 'x',
 'y']
```

- All of the global variables in `mymod` are attributes on the module object.
- What about the other attributes on the model object? Are those also global variables?

Yes, for at least some of them.

In [4]:
```
from importlib import reload
reload(mymod)
```

```
Welcome to mymod!
Thanks for visiting mymod!
```

```
Out[4]: <module 'mymod' from '/Users/reuven/Courses/Current/OReilly-Python-Oct2026-2026-10/mymod.py'>
```

```
In [5]: mymod.hello('abcd')
```

```
Out[5]: 'Hello, abcd, from mymod!'
```

What happens if your module imports 20 other modules, and one of them is taking a log time. Your entire startup time is now delayed becasuse of this one module.

Enter `lazy import`, as of 3.15 (a few weeks ago). If you say `lazy import`, then that object/module isn't reallyh loaded into memory. BUt the first time you use it, it will be.

```
In [6]: reload(mymod)
```

```
Welcome to mymod!
Thanks for visiting mymod!
```

```
Out[6]: <module 'mymod' from '/Users/reuven/Courses/Current/OReilly-Python-Oct2026-2026-10/mymod.py'>
```

`__name__` is very special

It always reflects the current *namespace* so you'll know if the module is running as a result of `import` or if it's running as a standalone.

The way it makes that distinction is with `__name__`:

- If the module was imported, then `__name__` is a string, the name of the module
- If the module is being run as a fullf-ledged program, then `__name__` is still a string, but is `'__main__'`.

How does this help us?

We can use an `if` statement in our module, and only print certain things when we're developing, not in production.

```
In [7]: reload(mymod)
```

```
Out[7]: <module 'mymod' from '/Users/reuven/Courses/Current/OReilly-Python-Oct2026-2026-10/mymod.py'>
```

When and how should we use this?

Just about every Python module uses this to keep things quiet when we're importing, but noisy/debugging when we run on the command line. That allows us to test things

more easily.

Under the `if` line only runs standalone. What can put there?

- documentation is displayed
- demos of the program
- self testing -- invoking the module triggers all tests on its contents

Exercise: Interactive menu

Modify `menu.py`, such that:

- If it is imported, say nothing (as now)
- If it is run directly, then the user should be able to choose from among three options: strawberry, peach, or apple

In [9]:
```
import menu
```

Modules that come with Python -- the "standard library"

Python comes with a very large number of modules, the "standard library." These aren't loaded automatically (you still have to `import` them), but they are guaranteed to be on a computer running Python.

The standard library is documented at https://docs.python.org/3/library/index.html .

For example, `glob` -- this module lets you search for filenames by pattern. You pass `glob.glob` a pattern, e.g., `'a*.txt'`. It'll pick all of the files that start with `a` and end with `.txt`.

Exercise: Using `glob` from the standard library

1. The `glob` module allows you to find files matching a pattern, using `*` and `?`.
2. Write a program that asks the user to enter a pattern.
3. Find all filenames matching that pattern.
4. For each filename, print the size of the file alongside the name. (Assume that the file contains only text, and you can then iterate over each line, adding to the total length.)

In [16]:
```
import glob

pattern = input('Enter pattern: ').strip()

for one_filename in glob.glob(pattern):
    size = 0
    for one_line in open(one_filename):
        size += len(one_line)

    print(f'{one_filename} : {size}')
```

```
OReilly-Python-Oct2026-2026-10-29.ipynb : 82958
OReilly-Python-Oct2026-2026-10-15.ipynb : 86390
OReilly-Python-Oct2026-2026-10-08.ipynb : 66880
OReilly-Python-Oct2026-2026-10-22.ipynb : 75565
OReilly-Python-Oct2026-2026-10-01.ipynb : 79264
```

Next up:

1. PyPI
2. `pip` and `uv`
3. Things to think about when using PyPI
4. AMA
5. Where to next?

Two scenarios for `import` and the whole `if __name__` stuff

- We have a module, `menu.py`. It contains two parts:
  - `def menu`, a function definition
  - `if __name__ == '__main__'`, followed by `print` in the block

Scenario 1:

- From another Python program, we say `import menu`
- Because a separate program is saying `import menu`, Python sets the `__name__` variable to be the string `'menu'` inside of `menu.py`.
- It executes `menu.py` in that context (i.e., `__name__` is now `'menu'`
- The first part of `menu.py` executes, because it's a `def` and has no guards on it
- The second part of `menu.py` does *NOT* execute, because it checks that `__name__ == '__main__'`, and that is *NOT* true here.
- The end result is that the `menu` function is defined and available to the caller, but nothing is printed or asked of the user.

Scenario 2:

- From the terminal, we invoke `menu.py` as a standalone program.
- Because we're invoking it as a standalon eprogram, Python sets the `__name__` variable to be the special string `'__main__'`
- It executes `menu.py` in that context (i.e., `__name__` is now `'__main__'`)
- The first part of `menu.py` executes, because it's a `def` and has no guards on it
- The second part of `menu.py` *DOES* execute, because it checks if `__name__ == '__main__'`, which IS TRUE NOW, because we're running the program standalone, i.e., not importing it
- The end result is not just that `menu` defines `menu.menu`, but it also shows with a brief demo the `menu` function in adction.

PyPI

The Python standard library is big, but it's still very small compared with the universe of things that people have created in the Python world.

Any module that is distributed with Python is in a bind -- it's very popular, and easy to use, but authors cannot update whenever they want. For this and some other reasons, we needed a repository for third-party packages in Pytohn. That's PyPI.

Python Package Index contains many *many* packages that you can download and install.

Downloading and installing

For years, the standard way to install packages from PyPI was a program called `pip` (Python installs package, or the like). You could say `pip install ABCD`, and it would look on PyPI for `ABCD`, and if it was there, it would download it and install it on your computer.

But... there were some issues. Let's say you're working on project A which needs mixing 1.0. You're also working on project B which needs mixing 2.0. Let's assume that each version of `mixing` is totally incompatible with others.

The traditional response is to use `venv`, but those are annoying.

`pip` knows how to download into a venv, which keeps different versions separate. But people were getting frustrated.

`uv` is the new darling of the Python world. You can use it to install packages, and to upload new packages to PyPI, too!

If you're using `pip` (the default), then installing something from PyPI is as easy as

```
pip install PACKAGENAME
```

If you're using uv, then you can say

```
uv add PACKAGENAME
```

Exercise: Use `rich` from PyPI

`rich` is a very popular project. It lets you write text in many different styles and colors

- Find `rich` on PyPI
- Use `pip` to install it. (You don't run `pip` inside of your Python environemnt, but rather in the shell)
- Write a short program that imports `rich` and displays a nicely formatted and/or colorized greeting.

Example:

```
import rich

rich.print('Hello! I [blue]am[/blue] a [yellow on black]sample greeting[/yellow on black].)
```

In [21]:
```
import rich

name = input('Enter your name: ').strip()

rich.print(f'Hello, [bold]{name}[/bold]! I [blue]am[/blue] a [yellow on blac
```

```
Hello, Reuven! I am a sample greeting.
```

What next?

- Practice, practice, and more practice. Find Python projects -- your own, and open source, too!
- More O'Reilly courses
  - Module of the week
  - What's happening in Python
  - Ask the expert
- Go to conferences and meetups -- and give a talk!

Contributing to open source

1. Find a project that is (a) interesting to you, (b) big enough to have a structure, (c) is run by nice people.
2. What kinds of contributions could you make?

In [ ]: